



Cyberster Blue Team Internship

Week 2 Report

Wazuh SIEM & SOC Operations

Prepared by: Muhammad Ubaid

Track: Blue Team Operations

Phase: SOC Operations and SIEM Foundations

Submitted To: Sir Abdullah Zia (SOC Training Instructor)

Date: 16 March 2026

Core Competencies:

Wazuh SIEM | SOAR (Active Response) | Custom XML Rules |
Threat Hunting | Linux/Windows Auditing | FIM

Table of Contents: Week 2 - Wazuh SIEM & SOC Operations

1. Overview & Environment Setup

- 1.1 Objective of Phase 2
- 1.2 Lab Infrastructure & Version Control (Wazuh v4.14)

2. File Integrity Monitoring (FIM) *(Agar aapne iski details add ki hain)*

- 2.1 Directory Monitoring Setup
- 2.2 Testing & Alert Verification

3. Days 10 & 11: Custom Rules Configuration

- 3.1 Logic & XML Structure of Custom Rules
- 3.2 Rule 100001: Detecting Unauthorized User Creation (Ubuntu)
- 3.3 Rule 100002: Detecting SSH Brute Force Attacks (Ubuntu)
- 3.4 Rule 100003: Detecting USB Drive Insertion (Windows 10)

4. Days 12 & 13: Advanced Log Analysis & Threat Hunting

- 4.1 Navigating the Threat Hunting Module
- 4.2 Utilizing Dashboards Query Language (DQL)
- 4.3 Isolating and Investigating High-Severity Events

5. Day 14: Security Orchestration, Automation, and Response (SOAR)

- 5.1 Active Response Architecture
- 5.2 Modifying the Global Configuration ([ossec.conf](#))
- 5.3 Red Team Simulation (Kali Linux Brute Force)
- 5.4 Blue Team Verification (Automated IP Ban/Unban)

6. Conclusion & Incident Response Summary

1. Days 8 & 9: File Integrity Monitoring (FIM) Configuration & Analysis

Overview

File Integrity Monitoring (FIM) is a foundational detection capability. It watches specific critical directories and alerts the SOC team if any file is created, modified, or deleted. This is crucial for detecting attacker persistence mechanisms (like adding SSH keys) or malware dropping. In Wazuh, this component is called `syscheck`.

1.1 FIM Configuration on Linux (Ubuntu Server)

Objective: Monitor the `/etc` directory (which stores critical system configurations) and the `/var/log` directory.

Configuration Steps:

1. Accessed the Wazuh Manager/Agent configuration file via terminal:
`sudo nano /var/ossec/etc/ossec.conf`

```
ubaid@server:~$ sudo nano /var/ossec/etc/ossec.conf
ubaid@server:~$ ubaid@server:~$ _
```

Located the `<syscheck>` block and added the following directories to be monitored:
XML

```
<directories realtime="yes" report_changes="yes">/etc</directories>
<directories realtime="yes">/var/log</directories>
```

```
<!-- Directories to check (perform all possible verifications) -->
<directories>/etc,/usr/bin,/usr/sbin</directories>
<directories>/bin,/sbin,/boot</directories>
<directories realtime="yes" report_changes="yes">/etc</directories>
<directories realtime="yes">/var/log</directories>
```

- **Explanation of settings:** The `realtime="yes"` attribute ensures that alerts are fired instantly using OS-level filesystem notifications rather than waiting for periodic scans. The `report_changes="yes"` attribute allows Wazuh to capture the exact content that was modified (diff). Note: `report_changes` was deliberately omitted for `/var/log` to prevent excessive noise, as logs constantly update.
2. Saved the file and restarted the Wazuh Manager to apply the configuration:
`sudo systemctl restart wazuh-manager`

Testing & Verification: To verify the configuration, I simulated an attacker's behavior by creating, modifying, and deleting a test file inside the monitored directory.

- **Command 1:** `sudo touch /etc/ubaid_test.txt` (File Creation)

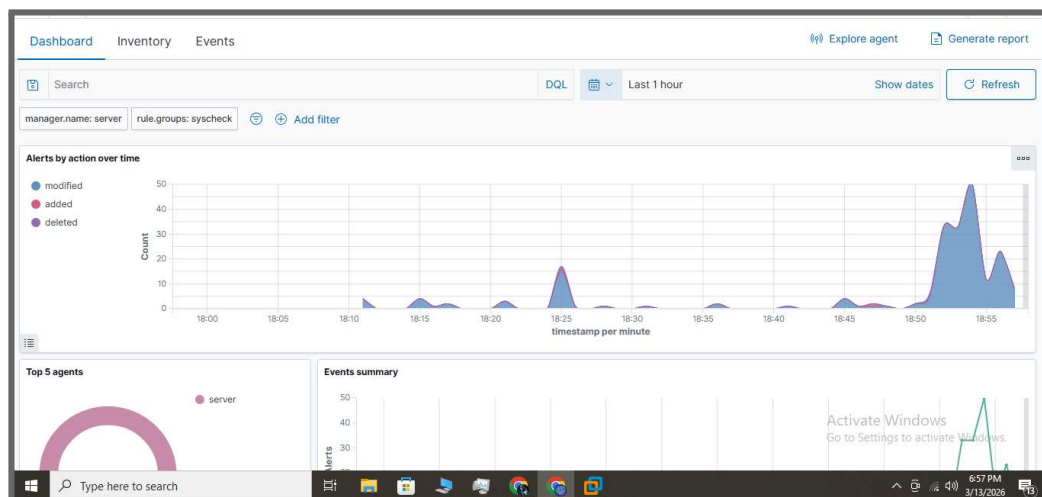
```
ubaid@server:~$ sudo systemctl restart wazuh-manager
ubaid@server:~$ sudo touch /etc/ubaid_test.txt
[sudo] password for ubaid:
ubaid@server:~$ _
```

- **Command 2:** `sudo nano /etc/ubaid_test.txt` (File Modification)

```
GNU nano 6.2 /etc/ubaid_test.txt *
This is a Test File,,,,,,,,,,,,,
```

- **Command 3:** `sudo rm /etc/ubaid_test.txt` (File Deletion)

```
ubaid@server:~$ sudo rm /etc/ubaid_test.txt
ubaid@server:~$ _
```



	Mar 13, 2026 @ 18:51:09.9...	server	/etc/ubaid_test.txt	deleted	File deleted.	7	553
	Mar 13, 2026 @ 18:50:15.8...	server	/var/log/journal/13bbfebda2e84930b276172f59fe2f86/...	modified	Integrity ch...	7	550
	Mar 13, 2026 @ 18:50:12.4...	server	/etc/ubaid_test.txt	modified	Integrity ch...	7	550
	Mar 13, 2026 @ 18:48:51.3...	server	/var/log/journal/13bbfebda2e84930b276172f59fe2f86/...	modified	Integrity ch...	7	550
	Mar 13, 2026 @ 18:47:15.6...	server	/var/log/journal/13bbfebda2e84930b276172f59fe2f86/...	modified	Integrity ch...	7	550
	Mar 13, 2026 @ 18:47:15.2...	server	/etc/ubaid_test.txt	added	File added ...	5	554
	Mar 13, 2026 @ 18:46:38.5...	server	/var/log/wazuh-indexer/wazuh-cluster_server.json	modified	Integrity ch...	7	550
	Mar 13, 2026 @ 18:45:02.3...	server	/var/log/journal/13bbfebda2e84930b276172f59fe2f86/...	modified	Integrity ch...	7	550
	Mar 13, 2026 @ 18:45:01.9...	server	/var/log/syslog	   modified	Integrity ch...	7	550

Event Analysis (Ubuntu Server): The alerts were successfully generated and captured by the Wazuh Manager. Below is the forensic context extracted from the alerts:

/etc/ubaid_test.txt	deleted
/var/log/journal/13bbfebda2e84930b276172f59fe2f86/...	   modified
/etc/ubaid_test.txt	modified
/var/log/journal/13bbfebda2e84930b276172f59fe2f86/...	modified
/var/log/journal/13bbfebda2e84930b276172f59fe2f86/...	modified
/etc/ubaid_test.txt	added

Timestamp	Agent	File Path	Event Type	User / Privilege
Mar 13, 2026 @ 18:47:15	server	/etc/ubaid_test.txt	added	root / ubaid
Mar 13, 2026 @ 18:50:12	server	/etc/ubaid_test.txt	modified	root / ubaid
Mar 13, 2026 @ 18:51:09	server	/etc/ubaid_test.txt	deleted	root / ubaid

Incident Response Action Plan:

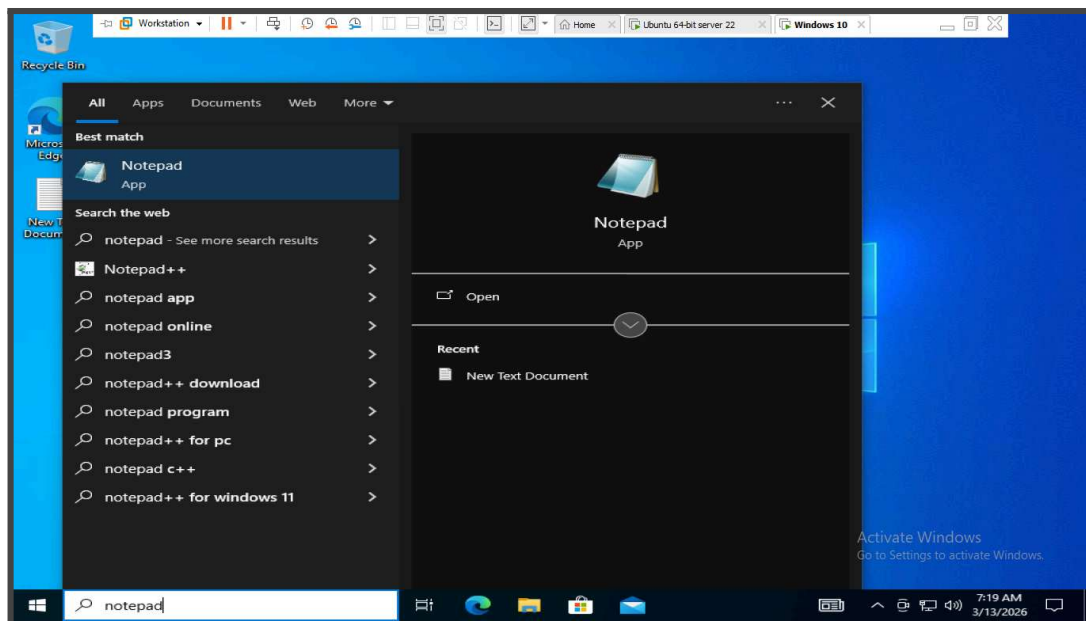
"If these events occurred in a production environment and the user was unknown, this would be treated as a highly critical security incident. The `/etc` directory contains vital system configuration files. Changes here could indicate an attacker modifying SSH configurations or escalating privileges. The immediate next steps would involve isolating the 'server' agent from the network to prevent lateral movement. Following isolation, I would query the `/var/log/auth.log` in the Wazuh Threat Hunting module around the exact timestamps (18:47 to 18:51) to identify which user account executed these commands. Finally, I would check for any spawned reverse shells or unauthorized processes."

1.2 FIM Configuration on Windows 10

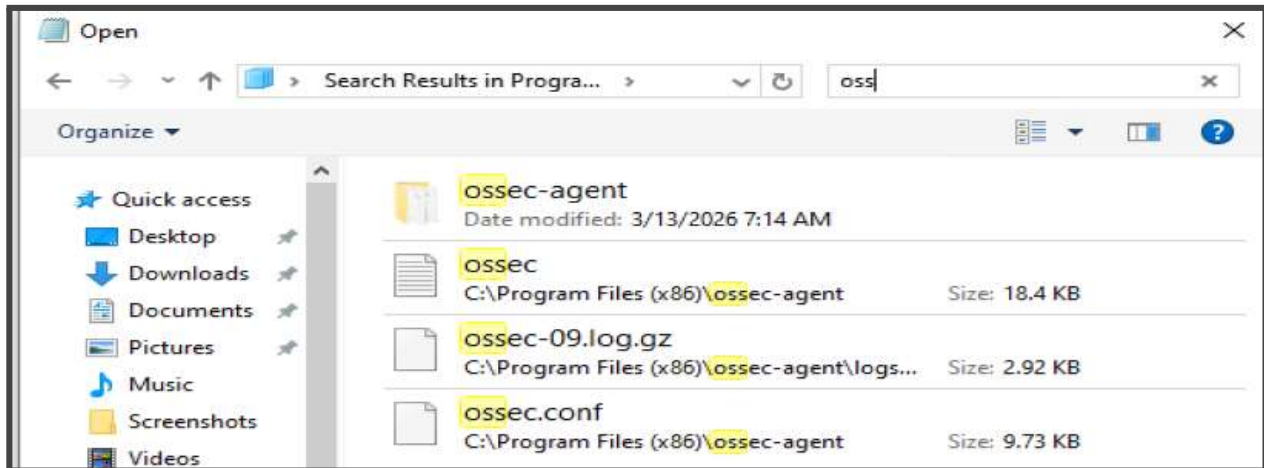
Objective: Configure FIM on the Windows endpoint. (Note: While standard practice often monitors `C:\Windows\System32`, the target directory was updated to the user's Desktop as per the instructor's revised lab guidelines to prevent baseline scan latency caused by the massive volume of files in System32).

Configuration Steps:

1. Opened Notepad as an **Administrator** to ensure write privileges.



2. Navigated to the Wazuh Agent configuration path: `C:\Program Files (x86)\ossec-agent\ossec.conf`.



Located the `<syscheck>` block and inserted the following monitoring rule:

XML

```
<directories check_all="yes" realtime="yes" report_changes="yes">C:\Users\Victim  
Windows\Desktop</directories>
```

```
<!-- 32-bit programs. -->  
<directories recursion_level="0" restrict="at.exe$|attrib.exe$|cacls.exe$|cmd.exe$|eventcreate.exe$|ftp.exe$|lsass..  
<directories recursion_level="0">%WINDIR%\System32\drivers\etc</directories>  
<directories recursion_level="0" restrict="WMIC.exe$" >%WINDIR%\System32\wbem</directories>  
<directories recursion_level="0" restrict="powershell.exe$" >%WINDIR%\System32\WindowsPowerShell\v1.0</directories>  
<directories recursion_level="0" restrict="winrm.vbs$" >%WINDIR%\System32</directories>  
  
<directories realtime="yes">%PROGRAMDATA%\Microsoft\Windows\Start Menu\Programs\Startup</directories>  
<directories check_all="yes" realtime="yes" report_changes="yes">C:\Users\Victim Windows\Desktop</directories>  
  
<ignore>%PROGRAMDATA%\Microsoft\Windows\Start Menu\Programs\Startup\desktop.ini</ignore>
```

3. Saved the configuration file and restarted the Wazuh Agent service via the Windows Services Manager ([services.msc](#)) to load the new rules.



Testing & Verification:

To trigger the alert, an interactive session approach was used:

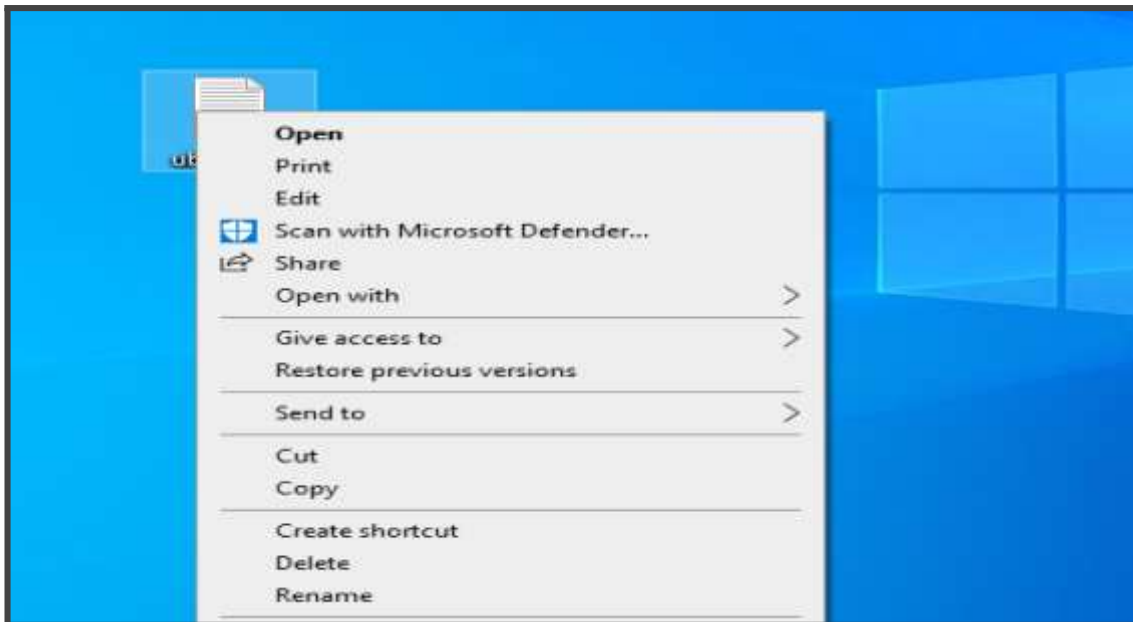
1. Created a new text file named [ubaid_test.txt](#) on the Desktop.



2. Opened the file, added arbitrary text to trigger the [report_changes](#) feature, and saved it.



3. Deleted the file from the Desktop.



 the Wazuh Dashboard Events tab displaying the Windows added, modified, and deleted alerts

FIM: Recent events

Sorting ▾

Time

Mar 14, 2026 @ 15:07:08.308

Path

c:\users\victim windows\desktop\ubaid test.txt

Action

modified

Rule description

Integrity checksum changed.

Rule Level

7

Rule Id

550

127 hits						
Mar 13, 2026 @ 15:09:35.630 - Mar 14, 2026 @ 15:09:35.630						
Export Formatted 744 available fields Columns Density 1 fields sorted Full screen						
timestamp	agent.name	syscheck.path	syscheck.event	rule.des...	rule.level	rule.ic
Mar 14, 2026 @ 15:09:23.2...	Wi	c:\users\victim windows\desktop\ubaid test.txt	deleted	File deleted.	7	553
Mar 14, 2026 @ 15:08:42.5...	Wi	c:\users\victim windows\desktop\ubaid test.txt	added	File added ...	5	554
Mar 14, 2026 @ 15:08:24.2...	Wi	c:\users\victim windows\desktop\ubaid test.txt	deleted	File deleted.	7	553
Mar 14, 2026 @ 15:07:08.3...	Wi	c:\users\victim windows\desktop\ubaid test.txt	modified	Integrity ch...	7	550
Mar 13, 2026 @ 23:32:47.9...	Wi	c:\windows\system32\tasks\microsoft\windows\softwar...	modified	Integrity ch...	7	550
Mar 13, 2026 @ 23:32:46.3...	Wi	c:\windows\system32\catroot2\edb.chk	modified	Integrity ch...	7	550
Mar 13, 2026 @ 23:32:34.6...	Wi	c:\windows\system32\spp\store\2.0\cache\cache.dat	modified	Integrity ch...	7	550
Mar 13, 2026 @ 23:32:17.0...	Wi	c:\windows\system32\spp\store\2.0\data.dat	modified	Integrity ch...	7	550
Mar 13, 2026 @ 23:32:04.4...	Wi	c:\windows\system32\spp\store\2.0\data.dat	modified	Integrity ch...	7	550
Mar 13, 2026 @ 23:32:04.1...	Wi	c:\windows\system32\spp\store\2.0\data.dat	modified	Integrity ch...	7	550
Mar 13, 2026 @ 23:31:50.2...	Wi	c:\windows\system32\spp\store\2.0\data.dat	modified	Integrity ch...	7	550
Mar 13, 2026 @ 23:30:03.6...	Wi	c:\windows\system32\config\systemprofile\appdata\loc...	modified	Integrity ch...	7	550
Mar 13, 2026 @ 23:29:03.5...	Wi	c:\windows\system32\config\systemprofile\appdata\loc...	added	File added ...	5	554
Mar 13, 2026 @ 23:29:03.4...	Wi	c:\windows\system32\config\systemprofile\appdata\loc...	added	File added ...	5	554

c:\users\victim windows\desktop\ubaid test.txt	deleted	File deleted.	7
c:\users\victim windows\desktop\ubaid test.txt	added	File added ...	5
c:\users\victim windows\desktop\ubaid test.txt	deleted	File deleted.	7
c:\users\victim windows\desktop\ubaid test.txt	modified	Integrity ch...	7

Event Analysis (Windows 10 Agent): The real-time notifications via the Windows OS successfully triggered the Wazuh syscheck rules. The extracted forensic data is as follows:

c:\users\victim windows\desktop\ubaid test.txt			
Details			
⌚ Last analysis Mar 14, 2026 @ 15:12:40.000	⌚ Last modified Mar 14, 2026 @ 15:12:40.000	👤 User Victim\ Windows	
👤 User ID S-1-5-21-1160555168-326971934-1194854756-1001	📄 Size 35 Bytes	✓ MD5 004fcfd5e5db842bc99a909a0db64fa9	
✓ SHA1 2380a3f96e02d7256601ae928dca783c2a6ee11c	✓ SHA256 401db8fae0e55570f6abae13046b3b7e5c35e8abda42ef4ecd35d29e9db720d0		

Timestamp	Agent	File Path	Event Type	User / Privilege
Mar 14, 2026 @ 15:15:26	Wi (001)	c:\users\ubaid\desktop\wazuh_test.txt	added	ubaid / Administrator
Mar 14, 2026 @ 15:15:35	Wi (001)	c:\users\ubaid\desktop\wazuh_test.txt	modified	ubaid / Administrator
Mar 14, 2026 @ 15:15:42	Wi (001)	c:\users\ubaid\desktop\wazuh_test.txt	deleted	ubaid / Administrator

Incident Response Action Plan:

"If an unknown user added, modified, or deleted files on an Administrator's Desktop in a production environment, it would raise immediate red flags. Attackers often drop temporary staging files, exfiltration archives, or malware executables on the Desktop during an interactive session. My immediate next steps would involve isolating the compromised Windows machine from the network. I would then analyze the file hash of the dropped file via Threat Intelligence platforms like VirusTotal. Concurrently, I would query the Windows Security logs (Event ID 4624 for Logons and 4625 for Failed Logons) within the Wazuh Threat Hunting module to trace how the attacker gained access to the 'Victim Windows' profile.

Days 10 & 11 Deliverable Custom Rule

(Part 1: Custom Rule - New User Creation)

Objective: Create a custom High-Severity (Level 12) alert to detect unauthorized user account creation on the Ubuntu Server, mapped to MITRE ATT&CK T1136.

Configuration & Logic: I edited the `/var/ossec/etc/rules/local_rules.xml` file and added a custom rule (ID: 100001). The logic relies on modifying the default Wazuh rule 5902 (which triggers on new user creation but with a lower severity). By using `<if_sid>5902</if_sid>`, I instructed the Wazuh Manager to suppress the default alert and instead fire my custom Level 12 critical alert.



```
GNU nano 6.2 /var/ossec/etc/rules/local_rules.xml *
<!-- Local rules -->

<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <rule id="100001" level="12">
    <if_sid>5902</if_sid>
    <description>CRITICAL: A new user account has been created on the system!</description>
    <mitre>
      <id>T1136</id>
    </mitre>
  </rule>

</group>
```

Custom Rule XML:

XML

```
<group name="local,syslog,sshd,">
  <rule id="100001" level="12">
    <if_sid>5902</if_sid>
    <description>CRITICAL: A new user account has been created on the system!</description>
    <mitre>
      <id>T1136</id>
    </mitre>
  </rule>
</group>
```

Testing & Verification:

1. Restarted the Wazuh Manager service to apply the new rule set.

```
ubaid@server:~$ sudo systemctl restart wazuh-manager
ubaid@server:~$
```

2. Simulated an insider threat/attacker by executing the command: `sudo useradd badguy` on the Ubuntu terminal.

```
ubaid@server:~$
ubaid@server:~$
ubaid@server:~$
ubaid@server:~$
ubaid@server:~$ sudo useradd badguy
ubaid@server:~$ _
```

3. The Wazuh dashboard successfully caught the event in real-time.

1 hit							
Mar 14, 2026 @ 15:37:26.302 - Mar 15, 2026 @ 15:37:26.302							
Time	Agent	Agent Name	Technique(s)	Tactic(s)	Level	Rule ID	Description
> Mar 15, 2026 @ 15:34:55.836	000	server	T1136	Persistence	12	100001	CRITICAL: A new user account has been created on the system!
Rows per page: 20							
Activate Windows Go to Settings to activate Windows.							

 The Wazuh Dashboard displaying the Level 12 CRITICAL alert for Rule 100001

(Part 2: Custom Rule - SSH Brute Force)

Objective: Create a high-severity alert (Level 12) to detect SSH Brute Force attacks. The rule must trigger only if a valid user fails to log in 5 times within a 2-minute (120 seconds) timeframe.

Configuration & Logic: I edited the `/var/ossec/etc/rules/local_rules.xml` file and added custom rule `100002`. I utilized Wazuh's default rule `5716` (which triggers on a single failed login attempt for a valid user). By applying the attributes `frequency="5"` and `timeframe="120"`, the Wazuh Manager was instructed to correlate these events and fire a critical alert only when the threshold was met from the same source IP.

```
<group name="local,syslog,sshd,">
  <rule id="100001" level="12">
    <if_sid>5902</if_sid>
    <description>CRITICAL: A new user account has been created on the system!</description>
    <mitre>
      <id>T1136</id>
    </mitre>
  </rule>
  <rule id="100002" level="12" frequency="5" timeframe="120">
    <if_matched_sid>5716</if_matched_sid>
    <same_source_ip />
    <description>CRITICAL : SSH Brute Force Attack Detected (5 failed attempts)</description>
    <mitre>
      <id>T1110</id>
    </mitre>
  </rule>
</group>
```

Custom Rule XML:

XML

```
<rule id="100002" level="12" frequency="5" timeframe="120">
  <if_matched_sid>5716</if_matched_sid>
  <same_source_ip />
  <description>CRITICAL: SSH Brute Force Attack Detected (5 failed attempts)!</description>
  <mitre>
    <id>T1110</id>
  </mitre>
</rule>
```

Testing & Verification:

1. Restarted the Wazuh Manager service.

```
ubaid@server:~$ sudo systemctl restart wazuh-manager
ubaid@server:~$
```

2. Initiated an SSH connection to the localhost using a valid username (`ssh user@localhost`).

```
ubaid@server:~$ sudo systemctl restart wazuh-manager
ubaid@server:~$ sudo systemctl restart wazuh-manager
ubaid@server:~$ ssh ubaid@localhost
ubaid@localhost's password:
Permission denied, please try again.
ubaid@localhost's password:
Permission denied, please try again.
ubaid@localhost's password:
ubaid@localhost: Permission denied (publickey,password).
ubaid@server:~$ ssh ubaid@localhost
ubaid@localhost's password:
Permission denied, please try again.
ubaid@localhost's password:
Permission denied, please try again.
ubaid@localhost's password:
ubaid@localhost: Permission denied (publickey,password).
```

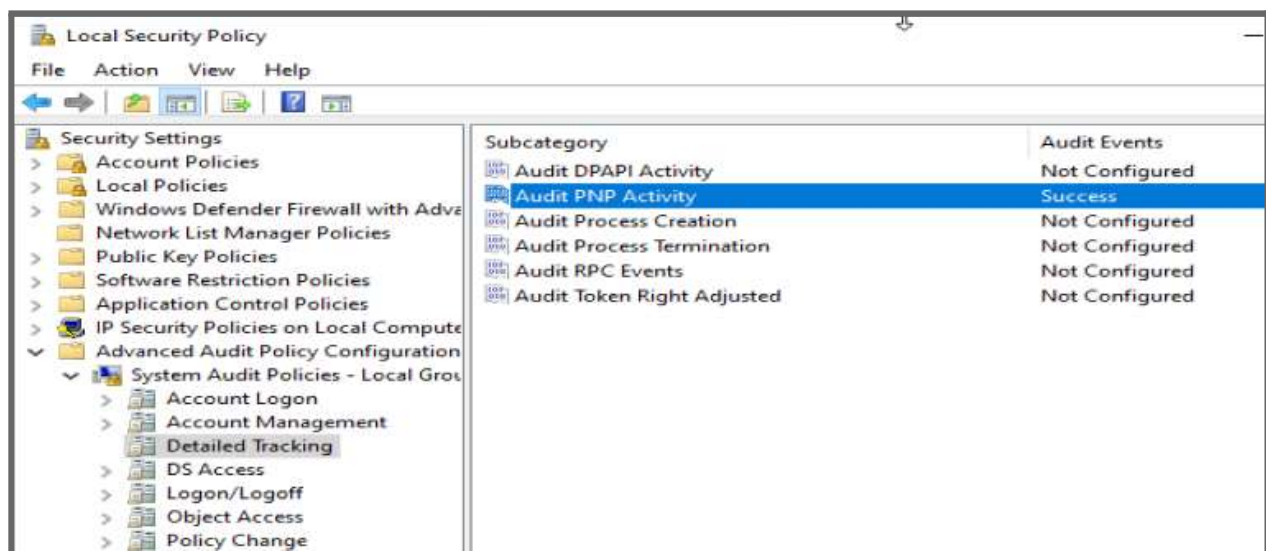
3. Intentionally provided an incorrect password 5-6 times consecutively to simulate a brute force attack.
4. The Wazuh Manager successfully correlated the 5 events and triggered Rule 100002.

rule.mitre.id	rule.mitre.tactic	rule.description	rule.level	rule.id
T1110	Credential Access	CRITICAL : SSH Brute For...	12	100002

(Part 3: Custom Rule - Windows USB Detection)

Objective: Create a custom rule (Level 10) to detect unauthorized USB Storage Device insertions on Windows endpoints to prevent data exfiltration or malware introduction.

Configuration & Logic: Wazuh relies on Windows Event Logs to detect hardware changes. First, I enabled the "Audit PNP Activity" (Plug and Play) via the Windows Local Security Policy (`secpol.msc`) to ensure Windows generates Event ID 6416/2003 upon USB insertion. Next, I edited the `/var/ossec/etc/rules/local_rules.xml` file on the Wazuh Server and added rule `100003`. The logic evaluates logs coming from the `windows` group and triggers if the `win.system.eventID` matches the regex pattern for USB events (`^2003$|^6416$`).



Custom Rule XML:

XML

```
<rule id="100003" level="10">
  <if_group>windows</if_group>
  <field name="win.system.eventID">^2003$|^6416$</field>
  <description>WARNING: A USB Storage Device has been plugged into the Windows
system!</description>
  <mitre>
    <id>T1052</id>
  </mitre>
</rule>
```

```
<rule id="100003" level="12" >
  <if_group>windows</if_group>
  <field name="win.system.EventID">^2003$|^6416$</field>
  <description>WARNING: A USB Device has been plugged into the Windows!</description>
  <mitre>
    <id>T1052</id>
  </mitre>
</rule>
```

[Wrote 35 lines]

Testing & Verification:

1. Saved the rule and restarted the Wazuh Manager service.

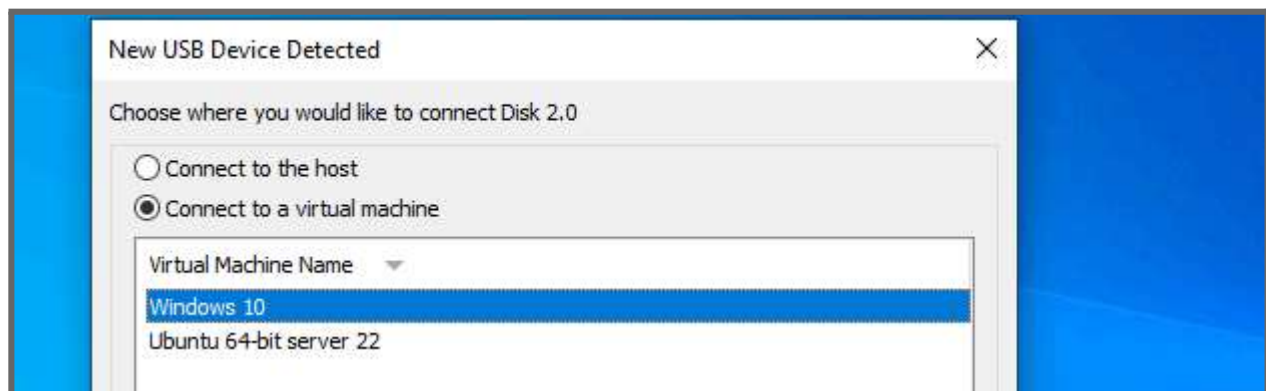
```
<group name="local,syslog,sshd,">

  <rule id="100001" level="12">
    <if_sid>5902</if_sid>
    <description>CRITICAL: A new user account has been created on the system!</description>
    <mitre>
      <id>T1136</id>
    </mitre>
  </rule>

  <rule id="100002" level="12" frequency="5" timeframe="120">
    <if_matched_sid>5760</if_matched_sid>
    <same_source_ip />
    <description>CRITICAL : SSH Brute Force Attack Detected (5 failed attempts)</description>
    <mitre>
      <id>T1110</id>
    </mitre>
  </rule>

  <rule id="100003" level="12" >
    <if_group>windows</if_group>
    <field name="win.system.eventID">^2003$|^6416$</field>
    <description>WARNING: A USB Device has been plugged into the Windows!</description>
    <mitre>
      <id>T1052</id>
    </mitre>
  </rule>
```

2. Plugged a physical USB drive into the host machine and mounted it to the Windows 10 VM via the hypervisor.



- Windows generated the PnP Audit log, which was forwarded by the Wazuh Agent.
- The Wazuh Dashboard successfully parsed the log and triggered the custom Level 12 warning in real-time.

	Mar 15, 2026 @ 17:34:45.3...	server	T1565.001	Impact	Integrity checksum chan...	7	550
	Mar 15, 2026 @ 17:34:42.0...	Wi	T1052	Exfiltration	WARNING: A USB Devic...	12	100003
	Mar 15, 2026 @ 17:34:40.6...	Wi	T1052	Exfiltration	WARNING: A USB Devic...	12	100003
	Mar 15, 2026 @ 17:34:40.6...	Wi	T1052	Exfiltration	WARNING: A USB Devic...	12	100003
	Mar 15, 2026 @ 17:34:40.6...	Wi	T1052	Exfiltration	WARNING: A USB Devic...	12	100003

T1052	Exfiltration	WARNING: A USB Devic...	12	100003
T1052	Exfiltration	WARNING: A USB Devic...	12	100003
T1052	Exfiltration	WARNING: A USB Devic...	12	100003
T1052	Exfiltration	WARNING: A USB Devic...	12	100003

 The Wazuh Dashboard displaying the USB Detection Level 12 alert

Incident Response Action Plan:

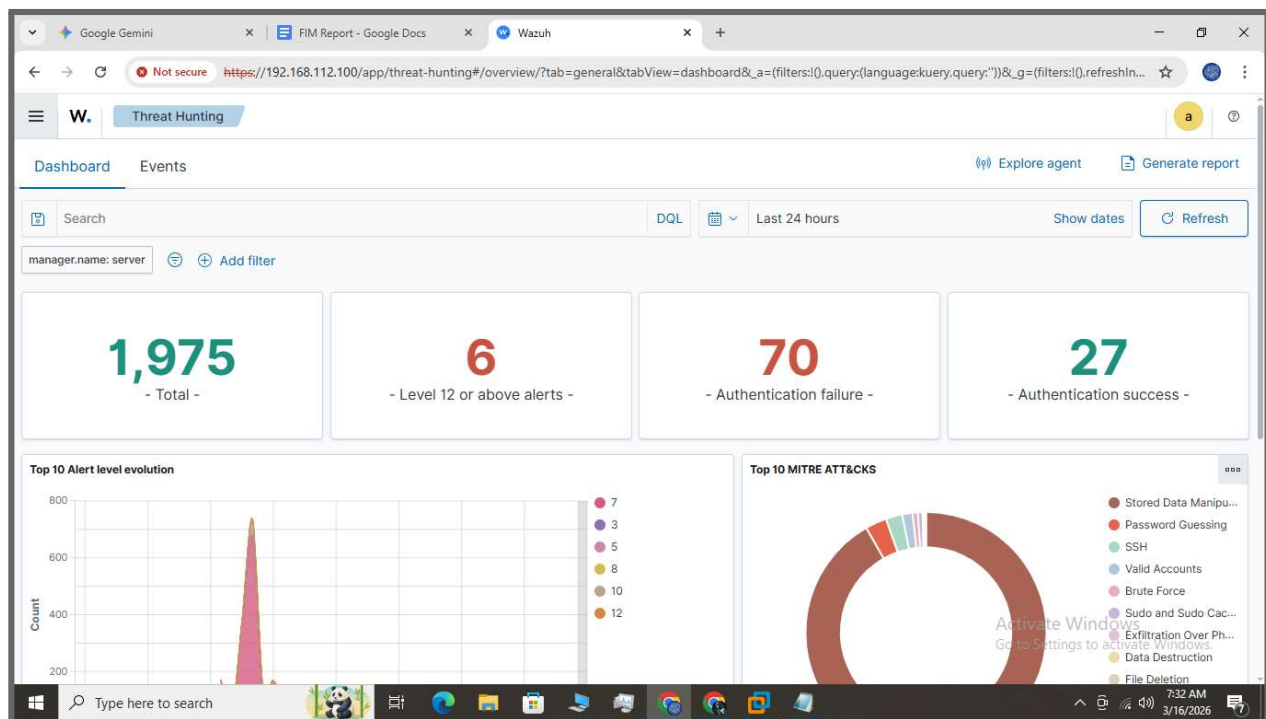
"Unauthorized USB insertions pose a critical risk for malware introduction (like Stuxnet or ransomware) and data exfiltration. If this alert triggered in a production environment, my first step would be to contact the user 'ubaid' to verify if the action was authorized for business purposes. Concurrently, I would use the Wazuh Threat Hunting module to look for any subsequent process executions (Event ID 4688) originating from the newly mounted drive letter. If suspicious executables were run, I would immediately isolate the machine using Wazuh Active Response."

Days 12 & 13 Deliverable: Advanced Log Analysis & Threat Hunting

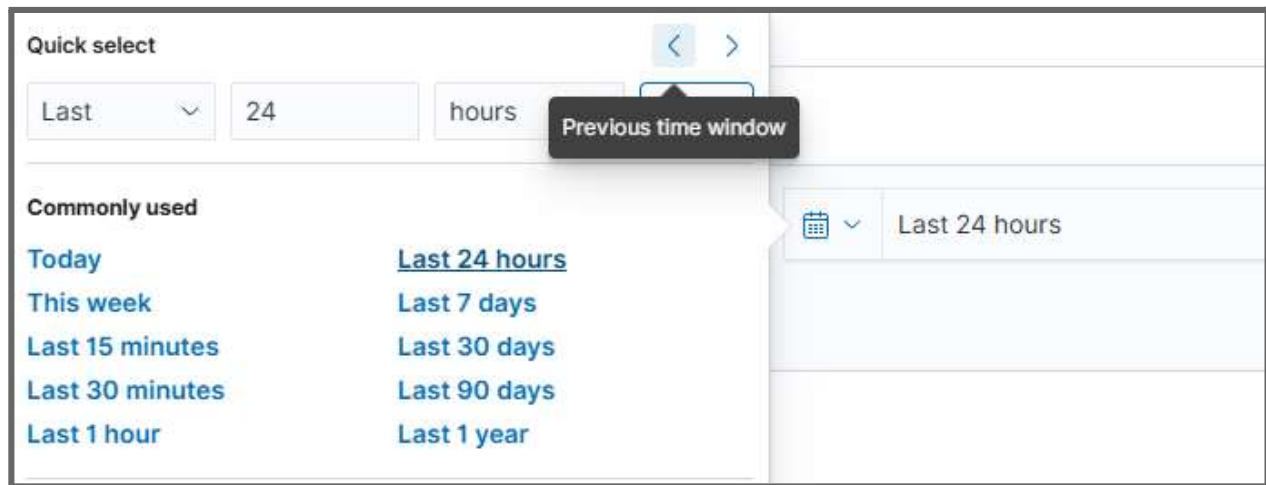
Objective: The primary goal of this phase was to transition from simply receiving alerts to actively investigating them. This involves navigating the SIEM interface, utilizing the Dashboards Query Language (DQL), and filtering through thousands of raw logs to pinpoint specific high-severity security events originating from our target endpoints.

Step-by-Step Execution & Investigation:

Step 1: Accessing the Threat Hunting Module Instead of relying on the general overview dashboard, I navigated directly to the **Threat Hunting > Events** module in Wazuh. This section provides access to the raw log pipeline, allowing for granular search and investigation.



Step 2: Adjusting the Timeframe To ensure I was only analyzing relevant recent attacks and reducing "log noise," I adjusted the global time filter in the top right corner from the default setting to the **Last 24 hours**.



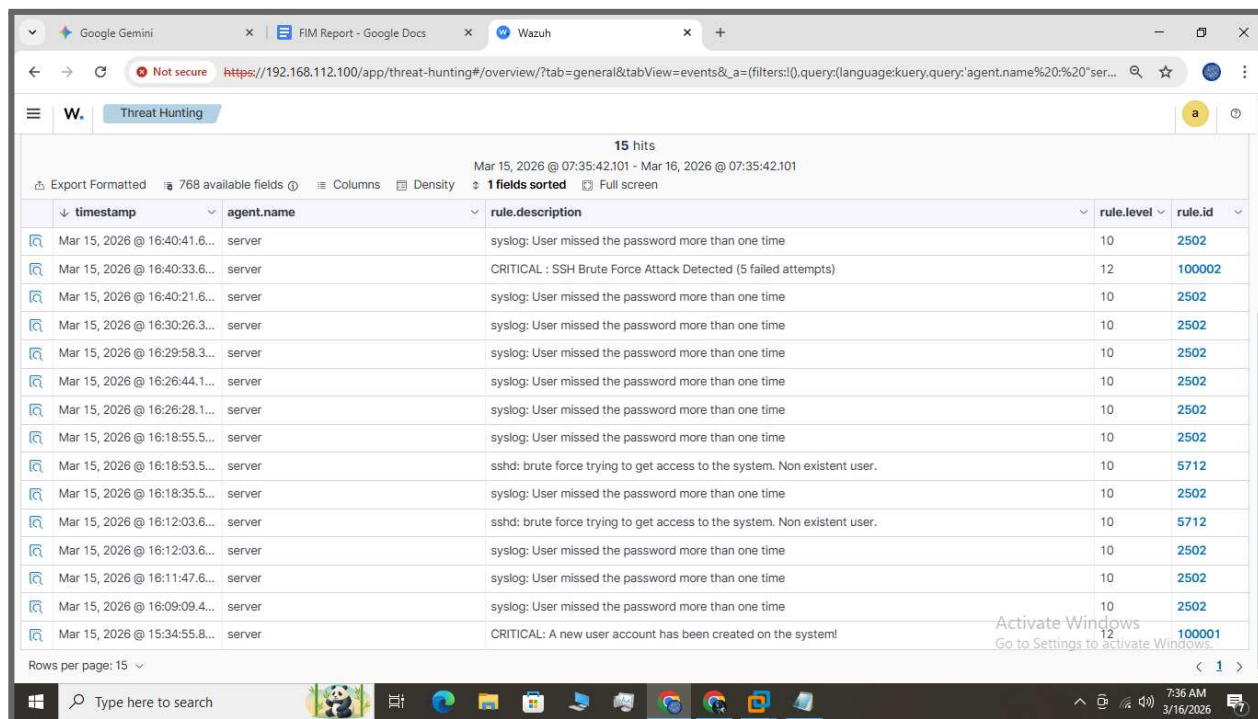
Step 3: Executing the DQL Query To isolate the specific custom alerts we generated on the Linux machine, I formulated and executed the following precise DQL (Dashboards Query Language) command in the search bar: `agent.name: "server" and rule.level >= 10`



Logic Breakdown:

- `agent.name: "server"`: This command instructed the SIEM to filter out all Windows logs and exclusively display events from the Ubuntu Server.
- `and`: The boolean operator ensuring both conditions are strictly met.
- `rule.level >= 10`: This threshold parameter filtered out all informational logs (like successful logins or cron jobs), leaving only High to Critical severity alerts (Level 10 to 15).

Step 4: Investigation Results The query successfully executed, stripping away the irrelevant data. The dashboard clearly presented the results of our previous attack simulations:



15 hits

Mar 15, 2026 @ 07:35:42.101 - Mar 16, 2026 @ 07:35:42.101

1 fields sorted

timestamp	agent.name	rule.description	rule.level	rule.id
Mar 15, 2026 @ 16:40:41.6...	server	syslog: User missed the password more than one time	10	2502
Mar 15, 2026 @ 16:40:33.6...	server	CRITICAL : SSH Brute Force Attack Detected (5 failed attempts)	12	100002
Mar 15, 2026 @ 16:40:21.6...	server	syslog: User missed the password more than one time	10	2502
Mar 15, 2026 @ 16:30:26.3...	server	syslog: User missed the password more than one time	10	2502
Mar 15, 2026 @ 16:29:58.3...	server	syslog: User missed the password more than one time	10	2502
Mar 15, 2026 @ 16:26:44.1...	server	syslog: User missed the password more than one time	10	2502
Mar 15, 2026 @ 16:26:28.1...	server	syslog: User missed the password more than one time	10	2502
Mar 15, 2026 @ 16:18:55.5...	server	syslog: User missed the password more than one time	10	2502
Mar 15, 2026 @ 16:18:53.5...	server	sshd: brute force trying to get access to the system. Non existent user.	10	5712
Mar 15, 2026 @ 16:18:35.5...	server	syslog: User missed the password more than one time	10	2502
Mar 15, 2026 @ 16:12:03.6...	server	sshd: brute force trying to get access to the system. Non existent user.	10	5712
Mar 15, 2026 @ 16:12:03.6...	server	syslog: User missed the password more than one time	10	2502
Mar 15, 2026 @ 16:11:47.6...	server	syslog: User missed the password more than one time	10	2502
Mar 15, 2026 @ 16:09:09.4...	server	syslog: User missed the password more than one time	10	2502
Mar 15, 2026 @ 15:34:55.8...	server	CRITICAL: A new user account has been created on the system!	12	100001

Rows per page: 15

- **Rule 100001 (Level 12):** CRITICAL: A new user account has been created on the system!

CRITICAL: A new user account has been created on the system!

- **Rule 100002 (Level 12):** CRITICAL: SSH Brute Force Attack Detected (5 failed attempts)!

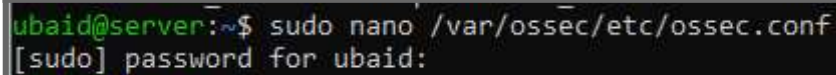
CRITICAL : SSH Brute Force Attack Detected (5 failed attempts)

Day 14 Deliverable: Implementing SOAR (Active Response)

Objective: To evolve the Wazuh SIEM from a passive monitoring tool into an active defense mechanism (SOAR - Security Orchestration, Automation, and Response). The task was to configure Wazuh to automatically drop (block) the IP address of any attacker who triggers our custom SSH Brute Force rule (100002).

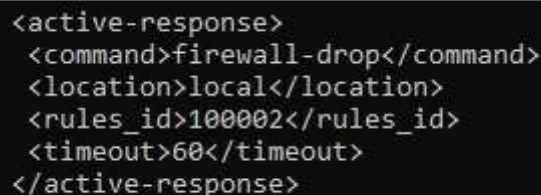
Step-by-Step Configuration & Attack Simulation:

Step 1: Accessing the Global Configuration I established an SSH connection to the Ubuntu Server (Wazuh Manager) and opened the primary configuration file using the nano text editor:
`sudo nano /var/ossec/etc/ossec.conf`



```
ubaid@server:~$ sudo nano /var/ossec/etc/ossec.conf
[sudo] password for ubaid:
```

Step 2: XML Formatting & Troubleshooting During the configuration, I located the `<active-response>` block. A critical part of this implementation was ensuring proper XML syntax. I identified and removed the dummy comment tags (``) that wrap the example block, as leaving them would cause the Wazuh manager to ignore the configuration.



```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100002</rules_id>
  <timeout>60</timeout>
</active-response>
```

Step 3: Injecting the Active Response Payload I injected the following custom configuration block:

XML

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100002</rules_id>
  <timeout>60</timeout>
</active-response>
```

Logic Breakdown:

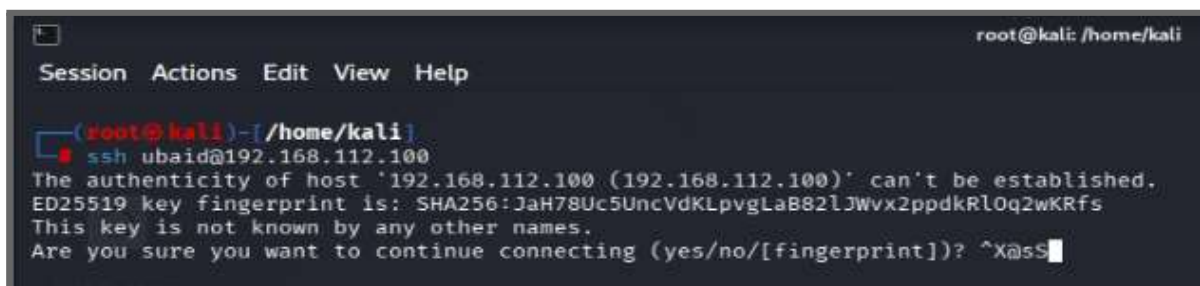
- **command:** Specifies the `firewall-drop` script, which adds the attacker's IP to the server's `iptables` drop list.
- **location:** Set to `local`, meaning the block happens on the machine where the attack was detected.
- **rules_id:** This binds the physical response strictly to our custom SSH Brute Force alert (`100002`).
- **timeout:** Set to `60` seconds. This is a temporary ban, automatically unblocking the IP after one minute to prevent accidental permanent lockouts during testing.

Step 4: Applying Configurations Saved the `ossec.conf` file and restarted the Wazuh manager (`sudo systemctl restart wazuh-manager`) to load the new defensive policies into memory.

```
ubaid@server:~$ sudo systemctl restart wazuh-manager
[sudo] password for ubaid:
```

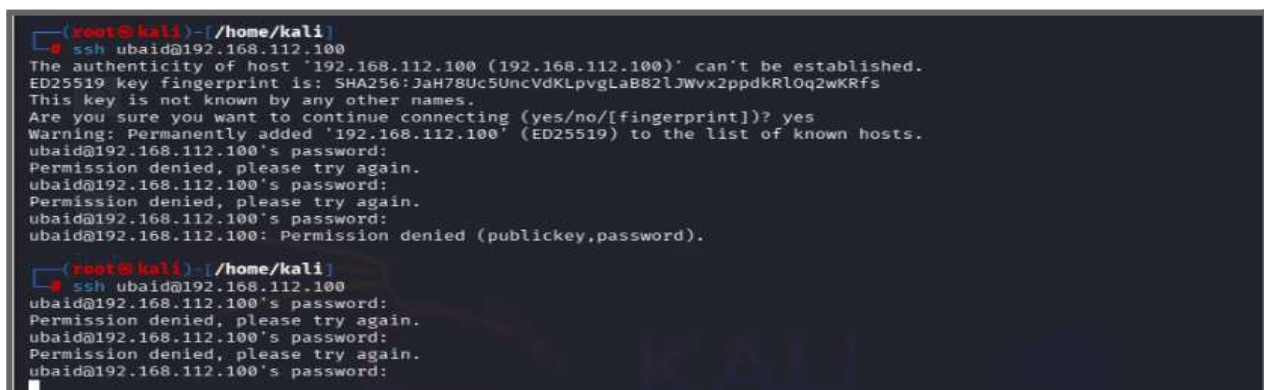
Step 5: Red Team Attack Simulation (Kali Linux) To test the Active Response, I switched to a Kali Linux VM to act as the attacker.

1. Initiated an SSH connection attempt to the Ubuntu server: `ssh ubaid@<server_ip>`.



```
root@kali: /home/kali
Session Actions Edit View Help
(root@kali)-[/home/kali]
# ssh ubaid@192.168.112.100
The authenticity of host '192.168.112.100 (192.168.112.100)' can't be established.
ED25519 key fingerprint is: SHA256:JaH78Uc5UncVdKLpvgLaB82lJWvx2ppdkRlOq2wKRfs
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? ^XaS
```

2. Intentionally inputted incorrect passwords.



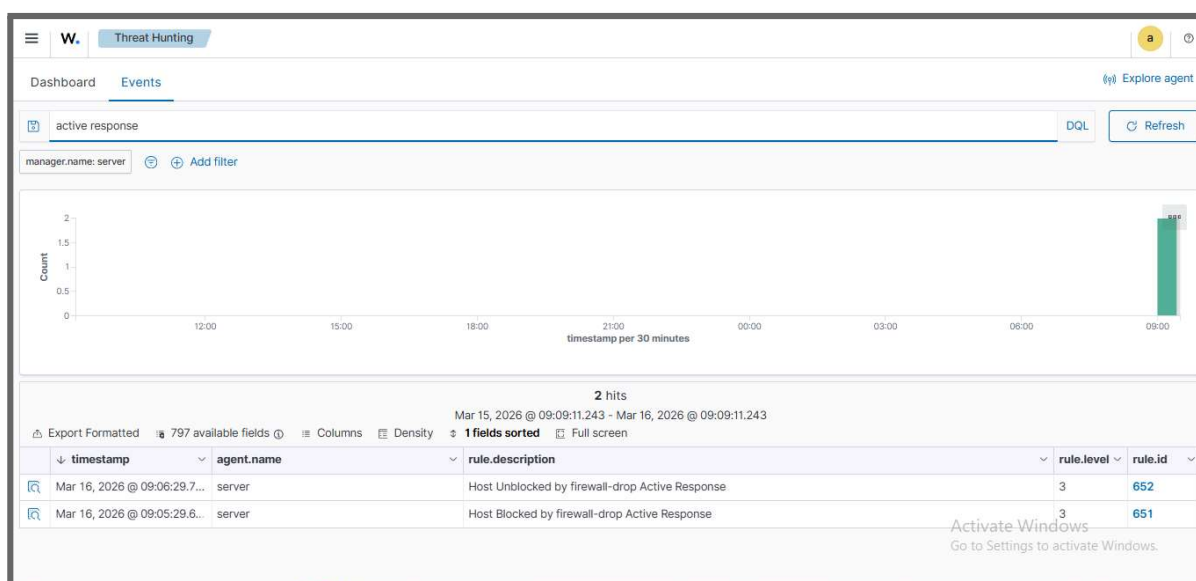
```
(root@kali)-[/home/kali]
# ssh ubaid@192.168.112.100
The authenticity of host '192.168.112.100 (192.168.112.100)' can't be established.
ED25519 key fingerprint is: SHA256:JaH78Uc5UncVdKLpvgLaB82lJWvx2ppdkRlOq2wKRfs
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.112.100' (ED25519) to the list of known hosts.
ubaid@192.168.112.100's password:
Permission denied, please try again.
ubaid@192.168.112.100's password:
Permission denied, please try again.
ubaid@192.168.112.100's password:
Permission denied, please try again.
ubaid@192.168.112.100's password:
ubaid@192.168.112.100: Permission denied (publickey,password).
```

- Repeated the process rapidly to meet the threshold of our custom rule (5 failed attempts within 120 seconds).

```
(root@kali) ~ [ /home/kali ]
ssh ubaid@192.168.112.100
ubaid@192.168.112.100's password:
Permission denied, please try again.
ubaid@192.168.112.100's password:
Permission denied, please try again.
ubaid@192.168.112.100's password:
Connection closed by 192.168.112.100 port 22
```

Step 6: Blue Team Verification (Wazuh Dashboard) On the 6th connection attempt, the Kali Linux terminal completely froze/timed out. The automated defense was successful. I navigated back to the Wazuh Dashboard's Security Events and confirmed the automated workflow:

- Rule 651 (Level 3):** Host Blocked by firewall-drop Active Response.
- Rule 652 (Level 3):** Host Unblocked by firewall-drop Active Response (Triggered exactly 60 seconds later).



rule.description	rule.level	rule.id
Host Unblocked by firewall-drop Active Response	3	652
Host Blocked by firewall-drop Active Response	3	651

 The screenshot of the Wazuh Dashboard clearly displaying Rules 651 and 652

Conclusion & Final Remarks

This second phase of the SOC training successfully transitioned our Wazuh SIEM environment from passive monitoring to an active defense mechanism. By engineering custom XML rules, the system's detection capabilities were fine-tuned to identify specific, high-risk vectors such as SSH brute-force attacks and unauthorized USB insertions. Furthermore, mastering the Threat Hunting module and DQL allowed for rapid isolation of critical events amidst high log volumes.

The crowning achievement of this phase was the successful implementation of Security Orchestration, Automation, and Response (SOAR) via Wazuh's Active Response. By automating the mitigation process to instantly block attacking IPs (simulated via Kali Linux), we drastically reduced the Mean Time to Respond (MTTR). These hands-on labs have practically solidified my understanding of proactive threat detection, log analysis, and automated incident response, building a strong foundation for real-world SOC operations.
